

Mammon V3 PRD - Part 1: Core Logic & System Architecture

1. Executive Summary & Strategy Pivot

Mammon V3 abandons passive, continuous market making. It is a **Micro-Momentum Tape Sniper** designed strictly for micro-capital (\$86 pool, trading \$5.10-\$6.00 tranches) on Binance 0% FDUSD promotional pairs.

The Sniper Workflow:

1. **Shadow Mode:** By default, the engine holds 0 open orders. It monitors the Limit Order Book (LOB) and Trade Flow Imbalance (TFI) in the background.
2. **The Trigger:** When Order Book Imbalance (OBI) heavily skews to the Buy side (indicating a massive retail or institutional wall), the engine attacks.
3. **The Penny-Jump (Entry):** It fires a LIMIT_MAKER order exactly 1 tick ahead of the Best Bid to cut the line and steal the fill.
4. **The Strike (Exit):** The millisecond the entry fills, it places a LIMIT_MAKER Ask at Average Entry Price (AEP) + Take_Profit_Ticks.
5. **The Amputation:** If the momentum reverses and drops below AEP - Stop_Loss_Ticks, it executes a Taker MARKET SELL to instantly free capital.

2. System Topology

The system is built as a strict tri-node architecture. Nodes communicate strictly via Redis and PostgreSQL.

```
mammon_v3/
├── execution_node/      # RUST (Ultra-low latency muscle)
│   ├── src/main.rs      # WSS Multiplexer & UDS Fill Tracker
│   ├── src/sniper_hft.rs # Ironclad State Machine (No AS Math)
│   └── src/binance_v3.rs # REST Client (Strict LIMIT_MAKER enforcement)
├── cognitive_node/      # PYTHON (The AI Brain)
│   ├── semantic_watcher.py # Dimensionality Reduction (Math -> Text)
│   ├── agent_q_loop.py    # LLM Orchestration & RAG Memory
│   └── prompts/           # Oracle, Alpha, and Risk Manager instructions
└── docker-compose.yml    # Boots Postgres, Redis, Rust, and Python
```

3. Database Schema (The Hippocampus)

We must persist the AI's parameter choices and the physical results to allow for Reinforcement

Learning (RL).

Target Schema (init_db.sql):

```
CREATE TABLE IF NOT EXISTS trade_telemetry (  
  id SERIAL PRIMARY KEY,  
  timestamp TIMESTAMPTZ DEFAULT NOW(),  
  symbol VARCHAR(20),  
  trade_size_usd FLOAT,  
  gross_pnl FLOAT,  
  fees_paid FLOAT,  
  net_pnl FLOAT  
);
```

```
CREATE TABLE IF NOT EXISTS agent_q_memory (  
  id SERIAL PRIMARY KEY,  
  timestamp TIMESTAMPTZ DEFAULT NOW(),  
  symbol VARCHAR(20),  
  regime VARCHAR(50),  
  -- V3 Sniper Levers (AI Decisions)  
  momentum_trigger_obi FLOAT,  
  take_profit_ticks INT,  
  stop_loss_ticks INT,  
  max_active_tranches INT,  
  -- Outcomes (Measured 3 minutes later)  
  total_round_trips INT DEFAULT 0,  
  win_rate_pct FLOAT DEFAULT 0.0,  
  net_pnl FLOAT DEFAULT 0.0,  
  reward_score FLOAT DEFAULT 0.0  
);
```

4. Key Learnings Implemented by Default

To prevent the failures of V1/V2, Mammon V3 assumes the following absolute truths:

1. **Never Average Up:** The bot is mathematically forbidden from acquiring a second tranche at a price higher than its current Average Entry Price (AEP).
2. **Dust Recovery:** If a partial fill traps inventory below Binance's \$5.00 limit, the bot must aggressively acquire the rest of the tranche before it is allowed to sell.
3. **Zero-Latency Fills:** Polling REST APIs for fills guarantees adverse selection. The Rust node MUST use the Binance WebSocket User Data Stream (UDS) for sub-10ms fill notifications.